

Python과 친해지기

Python을 직접 실행해 보면서 데이터 분석의 첫걸음을 시작합니다.



오늘의 목표

- ✓ Google Colab 사용하기
- ✓ Python 코드 실행하기
- ✓ 변수 사용하기
- ✓ 리스트 사용하기
- ✓ 평균 계산하기
- ✓ 간단한 그래프 그리기



```
>>> print("Hello Python")  
Hello Python
```



핵심 포인트

직접 코드를 실행해 보면서 Python이 어렵지 않다는 것을 느껴봅시다!



STEP 1 Google Colab 접속하기

Google Colab은 웹에서 Python 코드를 실행할 수 있는 환경입니다.

🌐 접속 방법

- 1 Chrome 브라우저를 실행합니다. 
- 2 Google 계정으로 로그인합니다.
※ 학교 계정 또는 개인 계정 사용 
- 3 아래 주소를 입력하여 Colab 사이트에 접속합니다.
<https://colab.research.google.com> 
- 4 “새 노트북”을 클릭하여 새 노트북을 생성합니다.
※ 새 Python 3 노트북이 열립니다. 
- 5 이제 Python 코드를 작성하고 실행할 준비가 완료되었습니다! 



Tip

Colab은 별도의 설치 없이 웹에서 바로 사용할 수 있어 언제 어디서나 실습할 수 있습니다.

1 Colab 사이트 접속 화면



2 새 노트북 화면 (예시)



확인 체크

Colab 접속 및 새 노트북 생성이 완료되었나요?

☐

STEP 2 첫 번째 Python 코드 실행

Colab에서 간단한 Python 코드를 입력하고 실행해 봅시다.

코드 입력하기

아래 코드를 새로운 셀에 입력하세요.



```
1 print("Hello Python")
```



실행 방법

셀 왼쪽의 ▶ 버튼을 클릭하거나
단축키 **Shift + Enter** 를 누르세요.



TIP

print() 함수는 괄호 안의 내용을 화면에 출력해 줍니다.
따옴표(“ ” 또는 ‘ ’) 안의 내용은 **문자열(String)**입니다.

실행 결과 확인하기

코드를 실행하면 아래와 같이 결과가 표시됩니다.



Hello Python



확인 체크

실행 결과가 동일하게 출력되었나요?

☐

```
print("Hello Python")
```



Hello Python

STEP 3 출력하기

`print()` 함수를 사용하면 화면에 원하는 내용을 출력할 수 있습니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 print("광운대학교")
2 print("소프트웨어학부")
3 print("데이터 분석")
```

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



```
광운대학교
소프트웨어학부
데이터 분석
```



알아두기

- `print()` 안의 내용은 작은따옴표(' ') 또는 큰따옴표(" ") 안에 작성합니다.
- 여러 줄을 출력하려면 `print()`를 여러 번 사용합니다.



실습 문제

자신의 정보를 출력해 보세요.

예) 이름, 학과, 학년

1. _____

2. _____

3. _____



TIP

따옴표 안의 내용이 그대로 화면에 출력됩니다.

따옴표도 함께 출력하고 싶으면 작은따옴표(' ') 안에 큰따옴표(" ")를 사용하세요.

```
print(' "안녕하세요"')
```



```
"안녕하세요"
```

STEP 4 Python 계산기 사용하기

Python은 사칙연산을 간단하게 수행할 수 있는 계산기 역할을 합니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 print(10 + 5)
2 print(10 - 5)
3 print(10 * 5)
4 print(10 / 5)
```

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



```
15
5
50
2.0
```



연산자 설명

연산자	의미	예시	결과
+	더하기	10 + 5	15
-	빼기	10 - 5	5
*	곱하기	10 * 5	50
/	나누기	10 / 5	2.0



실습 문제

다음 계산을 수행해 보세요.

① 100 + 50

결과: _____

② 300 - 120

결과: _____

③ 15 * 8

결과: _____

④ 400 / 5

결과: _____

★ TIP

- 계산 결과는 자동으로 화면에 출력됩니다.
- 나누기 연산(/)의 결과는 실수(소수)로 출력될 수 있습니다.

10 / 3



3.3333333333333335

STEP 5 변수 만들기

변수는 데이터를 저장하는 공간입니다. **한 번 저장하면 재사용할 수 있습니다.**

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.

```
1 name = "홍길동"
2 print(name)
```

설명

- name은 변수 이름입니다.
- "홍길동"은 문자열(데이터)입니다.
- = 는 데이터를 저장(할당)하는 연산자입니다.
- `print(name)`은 name에 저장된 값을 출력합니다.

★ TIP

- 변수 이름은 영문, 숫자, 언더스코어(_)를 사용할 수 있습니다.
- 변수 이름은 숫자로 시작할 수 없습니다. (예: 1name ✕)
- 사용하기 쉬운 이름을 짓는 것이 중요합니다!

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.

```
 홍길동
```

실습 문제

자신의 이름을 변수에 저장하고 출력해 보세요.

```
 1
```

좋은 변수 이름 예시

 name, student_name, myAge, grade2

피해야 할 변수 이름 예시

 1name, my age, if, for

STEP 6 변수 사용하기

여러 개의 변수를 선언하고 값을 저장한 후 출력해 봅시다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.

```
1 name = "홍길동"
2 major = "소프트웨어학부"
3 grade = 2
4
5 print(name)
6 print(major)
7 print(grade)
```



설명

- 변수는 데이터를 저장하는 공간입니다.
- = 기호는 '~에 저장한다'는 의미입니다.
- `print()`를 사용하면 변수에 저장된 값을 출력할 수 있습니다.

★ TIP

변수 이름 규칙

- ✓ 영문, 숫자, 언더스코어(_) 사용 가능
- ✓ 숫자로 시작할 수 없음 (예: 1name ×)
- ✓ 대소문자 구분 (예: Name과 name은 다름)
- ✓ 예약어(if, for, while 등)는 사용할 수 없음

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



```
홍길동
소프트웨어학부
2
```

✓ 실습 문제

아래 정보를 변수에 저장하고 출력해 보세요.

항목	값
이름	김철수
학과	데이터사이언스학과
학년	3

※ 변수 이름은 자유롭게 정하세요.



좋은 변수 이름 예시

student_name, age, major, total_score



피해야 할 변수 이름 예시

2name, my name, class, print

STEP 7 리스트 사용하기

리스트(list)는 여러 개의 값을 순서대로 저장하는 자료구조입니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 numbers = [10, 20, 30, 40, 50]
2 print(numbers)
3 print(numbers[0])
4 print(numbers[2])
5 print(numbers[-1])
6 print(len(numbers))
```



설명

- `[ ]` 안에 여러 값을 **쉼표(,)**로 구분하여 저장합니다.
- **인덱스(index)**는 0부터 시작합니다.
(첫 번째 값의 인덱스는 0)
- `-1`은 마지막 값, `-2`는 마지막에서 두 번째 값을 의미합니다.
- `len(리스트)` 함수는 리스트의 길이(요소 개수)를 반환합니다.

★ TIP

- 리스트의 각 값은 인덱스를 사용하여 접근합니다. (예: `numbers[0]`, `numbers[1]`)
- 음수 인덱스를 사용하면 뒤에서부터 접근할 수 있습니다.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



```
[10, 20, 30, 40, 50]
10
30
50
5
```



실습 문제

다음 리스트를 사용하여 물음에 답하세요.

```
scores = [85, 90, 78, 92, 88]
```

- | | |
|---------------------|-----------|
| ① scores 전체를 출력하세요. | 결과: _____ |
| ② 첫 번째 점수를 출력하세요. | 결과: _____ |
| ③ 세 번째 점수를 출력하세요. | 결과: _____ |
| ④ 마지막 점수를 출력하세요. | 결과: _____ |
| ⑤ 점수의 개수를 출력하세요. | 결과: _____ |

```
numbers = [10, 20, 30, 40, 50]
```

인덱스	0	1	2	3	4
음수 인덱스	-5	-4	-3	-2	-1

STEP 9 반복문 사용하기

반복문(loop)을 사용하면 같은 작업을 여러 번 자동으로 수행할 수 있습니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 scores = [85, 90, 78, 92, 88]
2 total = 0
3 for score in scores:
4     total = total + score
5 average = total / len(scores)
6 print("평균:", average)
```



설명

- `for` 반복문은 정해진 횟수만큼 코드를 반복 실행합니다.
- `for 변수명 in 리스트:` 구조를 사용합니다.
- 들여쓰기를 통해 반복할 코드를 지정합니다.
- 반복을 이용하면 긴 코드를 간단하게 작성할 수 있습니다.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



평균: 86.6

코드 해설

- ① 리스트 `scores`의 각 값을 차례대로 `score`에 저장합니다.
- ② `total = total + score`를 반복하여 모든 점수를 더합니다.
- ③ `len(scores)`는 리스트의 길이(요소 개수)를 반환합니다.
- ④ 평균 = 총합 / 개수 를 계산하여 출력합니다.

실습 문제

다음 리스트의 평균을 구하여 출력해 보세요.

```
scores = [70, 60, 80, 90, 100]
```

결과: _____

★ TIP

- 반복문은 데이터 분석에서 매우 자주 사용됩니다.
- 리스트의 모든 값을 합하거나, 최대값/최소값을 찾을 때 유용합니다.
- `range()` 함수와 함께 사용하면 특정 횟수만큼 반복할 수 있습니다.

range() 예시

```
for i in range(5):
    print(i)
```



출력 결과

```
0
1
2
3
4
```


STEP 10 조건문 사용하기

조건문(if)을 사용하면 조건에 따라 다른 작업을 수행할 수 있습니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 score = 85
2 if score >= 90:
3     print("A학점입니다.")
4 elif score >= 80:
5     print("B학점입니다.")
6 elif score >= 70:
7     print("C학점입니다.")
8 else:
9     print("F학점입니다.")
```



설명

- 조건문의 기본 구조를 연습합니다.
- 비교 연산자: > (이상), < (미만), == (같음), != (다름)
- 여러 조건 중 처음으로 참(True)인 조건의 블록이 실행됩니다.
- 점수에 따라 학점을 출력하는 프로그램입니다.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



B학점입니다.

코드 해설

- if : 주어진 조건이 참(True)인 경우 실행
- elif : 이전 조건이 거짓(False)일 때 다음 조건 확인
- else : 모든 조건이 거짓(False)일 때 실행
- 들여쓰기는 조건문 범위를 나타냅니다.

실습 문제

다음 점수에 대해 학점을 출력해 보세요.

score = 92

결과: _____



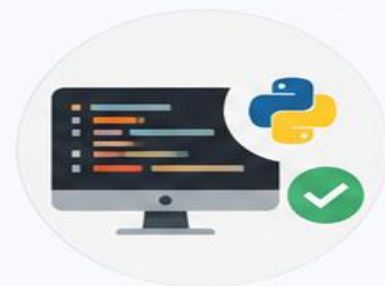
TIP

비교 연산자 정리

연산자	의미	예시 (score = 85)	결과
>	초과(보다 큼)	score > 90	False
>=	이상(크거나 같음)	score >= 80	True
<	미만(보다 작음)	score < 70	False
==	같음	score == 85	True
!=	같지 않음	score != 90	True

등급 기준 예시

점수 범위	학점
90 이상	A
80 이상 ~ 90 미만	B
70 이상 ~ 80 미만	C
70 미만	F



STEP 11 함수 만들기과 사용하기

함수(function)를 만들어 코드를 구조화하고 재사용성을 높여 봅니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.



```
1 def calculate_area(width, height):
2     area = width * height
3     return area
4
5
6 # 함수 호출
7 w = 5
8 h = 3
9 result = calculate_area(w, h)
10 print("가로: ", w, "세로: ", h)
11 print("면적: ", result)
```



설명

- 함수는 특정 작업을 수행하는 코드 블록을 이름으로 묶어 놓은 것입니다.
- 매개변수(parameter)는 함수에 값을 전달할 때 사용하는 변수입니다.
- return 키워드를 사용하면 함수의 결과 값을 반환합니다.
- 함수는 코드의 가독성과 재사용성을 높여줍니다.



TIP

- 함수 정의는 한 번만 하면 되고, 필요한 곳에서 여러 번 호출할 수 있습니다.
- 함수 이름은 의미 있게 작성하는 것이 좋습니다.
- 매개변수는 0개 이상 여러 개도 가능합니다.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



가로: 5 세로: 3
면적: 15

코드 해설

- def 키워드를 사용해 함수를 정의합니다.
- calculate_area(width, height)는 두 매개변수를 받아 넓이(면적)를 계산합니다.
- return 문을 사용해 계산한 값을 함수 호출 부분으로 반환합니다.
- 함수를 호출할 때는 함수명 뒤에 괄호() 안에 값을 전달합니다.
- 함수는 여러 번 호출하여 재사용할 수 있습니다.

실습 문제

다음 요구사항에 맞는 함수를 작성하고 호출하여 결과를 출력해 보세요.

두 수(a, b)를 매개변수로 받아 더한 값을 반환하는 함수 create_sum을 만드세요.

- ① create_sum(10, 20)을 호출하여 결과를 출력하세요. 결과: _____
- ② create_sum(-5, 8)을 호출하여 결과를 출력하세요. 결과: _____
- ③ create_sum(100, -30)을 호출하여 결과를 출력하세요. 결과: _____

함수 정의 형태

```
def 함수이름(매개변수1, 매개변수2, ...):
    실행할 코드
    return 반환할값
```

함수 호출 형태

결과 = 함수이름(인수1, 인수2, ...)

STEP 12 파일 입출력

텍스트 파일을 읽고 쓰는 방법을 실습합니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.

```
1 # 파일에 쓰기
2 with open("data.txt", "w", encoding="utf-8") as f:
3     f.write("이름,나이,점수\n")
4     f.write("홍길동,20,85\n")
5     f.write("김철수,22,90\n")
6     f.write("이영희,21,88\n")
7
8 # 파일에서 읽기
9 with open("data.txt", "r", encoding="utf-8") as f:
10     lines = f.readlines()
11     for line in lines:
12         print(line.strip()) # 줄 끝의 개행문자 제거
```



설명

- 파일 입출력은 데이터를 저장하고 불러오는 데 필수적입니다.
- "w" 모드로 열면 파일이 없으면 새로 생성되고, 있으면 덮어씁니다.
- "a" 모드는 기존 내용 뒤에 추가로 씁니다.
- 인코딩을 지정해야 한글이 깨지지 않습니다.



TIP

- 경로는 현재 작업 폴더 기준입니다. 필요 시 전체 경로를 사용하세요.
- with 문을 사용하면 파일을 자동으로 닫아 안전합니다.
- 파일을 다룰 때는 모드("r", "w", "a")를 정확히 지정하세요.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



```
이름,나이,점수
홍길동,20,85
김철수,22,90
이영희,21,88
```

코드 해설

- open("파일명", "모드", encoding="utf-8") : 파일을 엽니다.
- "w" : 쓰기 모드 (기존 파일이 있으면 내용을 덮어씁니다.)
- "r" : 읽기 모드
- readlines() : 파일의 모든 줄을 리스트로 읽어옵니다.
- strip() : 줄 끝의 줄바꿈 문자(\n)를 제거합니다.

실습 문제

- 1) 주어진 students.txt 파일을 읽어 각 학생의 이름과 점수를 출력하세요.
- 2) scores.txt 파일을 만들어 다음 점수를 한 줄에 하나씩 저장하세요.
[95, 88, 76, 91, 83]
- 3) scores.txt 파일을 읽어 모든 점수의 평균을 계산하여 출력하세요.

주요 파일 모드

모드	설명	기존 파일이 없을 때	기존 파일이 있을 때
r	읽기	오류 발생	읽기 전용으로 열기
w	쓰기	새 파일 생성	내용을 지우고 새로 씀
a	추가	새 파일 생성	내용 뒤에 추가
r+	읽기/쓰기	오류 발생	읽기/쓰기 모드로 열기

STEP 13 리스트 기본 연산

리스트의 인덱싱, 슬라이싱, 결합, 반복 등 기본 연산을 실습합니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.

```
1 # 리스트 생성
2 fruits = ["apple", "banana", "cherry", "orange"]
3 print("전체 리스트:", fruits)
4
5 # 인덱싱
6 print("첫 번째 원소:", fruits[0])
7 print("마지막 원소:", fruits[-1])
8
9 # 슬라이싱
10 print("두 번째부터 세 번째까지:", fruits[1:3])
11
12 # 리스트 결합과 반복
13 more = ["kiwi", "mango"]
14 print("결합:", fruits + more)
15 print("반복 (fruits * 2):", fruits * 2)
```



설명

- 리스트는 여러 값을 순서대로 저장하는 자료구조입니다.
- 인덱스는 0부터 시작합니다.
- 슬라이싱을 이용하면 연속된 여러 값을 쉽게 가져올 수 있습니다.
- 리스트는 다른 리스트와 결합하거나 반복할 수 있습니다.

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



전체 리스트: ['apple', 'banana', 'cherry', 'orange']
 첫 번째 원소: apple
 마지막 원소: orange
 두 번째부터 세 번째까지: ['banana', 'cherry']
 결합: ['apple', 'banana', 'cherry', 'orange', 'kiwi', 'mango']
 반복 (fruits * 2): ['apple', 'banana', 'cherry', 'orange', 'apple', 'banana', 'cherry', 'orange']

코드 해설

- 인덱싱: 리스트명[인덱스]로 특정 위치의 원소에 접근합니다.
- 음수 인덱스: -1은 마지막 원소, -2는 마지막에서 두 번째 원소입니다.
- 슬라이싱: 리스트명[시작:끝]은 시작 인덱스부터 끝-1 인덱스까지 가져옵니다.
- 결합: + 연산자로 리스트를 연결합니다.
- 반복: * 연산자로 리스트를 여러 번 반복합니다.

실습 문제

아래 리스트를 이용하여 물음에 답하고 결과를 출력해 보세요.

```
nums = [10, 20, 30, 40, 50]
```

- ① 첫 번째 원소를 출력하세요.
- ② 마지막 원소를 출력하세요.
- ③ 세 번째부터 네 번째까지의 원소를 출력하세요.
- ④ [60, 70]을 결합한 결과를 출력하세요.
- ⑤ 리스트를 3번 반복한 결과를 출력하세요.

★ TIP

인덱싱

```
fruits[2] → 'cherry'
fruits[-2] → 'cherry'
```

슬라이싱

```
fruits[1:3] → ['banana', 'cherry']
fruits[:2] → ['apple', 'banana']
fruits[2:] → ['cherry', 'orange']
```

결합

```
fruits + more
→ ['apple', 'banana', 'cherry', 'orange', 'kiwi', 'mango']
```

반복

```
fruits * 2
→ ['apple', 'banana', 'cherry', 'orange', 'apple', 'banana', 'cherry', 'orange']
```



STEP 14 딕셔너리 기본 연산

키-값 쌍으로 데이터를 저장하는 딕셔너리의 기본 연산을 실습합니다.

코드 입력하기

아래 코드를 새로운 셀에 입력하고 실행해 보세요.

```
1 # 딕셔너리 생성
2 student = {'이름': '홍길동', '나이': 20, '점수': 85}
3 print("전체 딕셔너리:", student)
4
5 # 값 접근
6 print("이름:", student['이름'])
7 print("점수:", student['점수'])
8
9 # 값 추가/수정
10 student['학과'] = '컴퓨터공학과'
11 student['점수'] = 90
12 print("추가/수정 후:", student)
13
14 # 키 삭제
15 del student['나이']
16 print("삭제 후:", student)
```



설명

- 딕셔너리는 순서가 없는 키-값 쌍의 모음입니다.
- 키(key)는 고유해야 하며 변경할 수 없는 자료형(문자열, 숫자, 튜플 등)이 사용됩니다.
- 값(value)은 모든 자료형을 사용할 수 있습니다.
- 효율적인 데이터 조회, 추가, 삭제가 가능합니다.

★ TIP

- 딕셔너리는 키를 이용해 빠르게 값을 찾을 수 있습니다.
- `in` 연산자로 키 존재 여부를 확인할 수 있습니다.
예) `'수학' in data → True/False`
- `keys()`, `values()`, `items()` 메서드를 사용하면 키 목록, 값 목록, (키, 값) 쌍을 쉽게 얻을 수 있습니다.

주요 딕셔너리 연산

연산	예시	설명
값 접근	<code>d['키']</code>	키에 해당하는 값을 반환
값 추가/수정	<code>d['새키'] = 값</code>	새 키 추가 또는 기존 값 수정
키 삭제	<code>del d['키']</code>	지정된 키와 값 삭제
키 존재 확인	<code>'키' in d</code>	키가 딕셔너리에 존재하면 True
모든 키	<code>d.keys()</code>	모든 키를 반환
모든 값	<code>d.values()</code>	모든 값을 반환
모든 항목	<code>d.items()</code>	(키, 값) 쌍을 반환

실행 결과 확인하기

실행하면 아래와 같이 결과가 출력됩니다.



전체 딕셔너리: {'이름': '홍길동', '나이': 20, '점수': 85}
 이름: 홍길동
 점수: 85
 추가/수정 후: {'이름': '홍길동', '나이': 20, '점수': 90, '학과': '컴퓨터공학과'}
 삭제 후: {'이름': '홍길동', '점수': 90, '학과': '컴퓨터공학과'}

i 코드 해설

- 딕셔너리는 중괄호 {} 안에 키: 값 쌍을 쉼표로 구분하여 저장합니다.
- 키를 사용하여 값을 조회할 수 있습니다: `student['이름']`
- 존재하지 않는 키를 조회하면 `KeyError`가 발생합니다.
- 새로운 키를 추가하거나 기존 값은 수정할 수 있습니다.
- `del`을 사용하여 특정 키와 그 값을 삭제할 수 있습니다.

✓ 실습 문제

다음 딕셔너리 `data`를 사용하여 질문에 답하고 결과를 출력해 보세요.

```
data = {'국어': 80, '영어': 90, '수학': 85}
```

- | | |
|----------------------------------|-----------|
| ① '영어' 과목의 점수를 출력하세요. | 결과: _____ |
| ② '과학' 과목의 점수 95를 추가하세요. | 결과: _____ |
| ③ '국어' 점수를 88로 수정하세요. | 결과: _____ |
| ④ '수학' 과목을 삭제하세요. | 결과: _____ |
| ⑤ 남아있는 모든 과목의 평균 점수를 계산하여 출력하세요. | 결과: _____ |

딕셔너리 구조 예시



STEP 15

Mini Project
나의 성적 분석

지금까지 배운 내용을 활용하여 나의 성적을 분석해 봅시다.



문제

다음 성적 데이터를 이용하여 분석하시오.

```
scores = [85, 90, 78, 92, 88]
```



수행 내용

① 평균 계산

```
avg = sum(scores) / len(scores)
```

② 최고점 계산

```
max_score = max(scores)
```

③ 최저점 계산

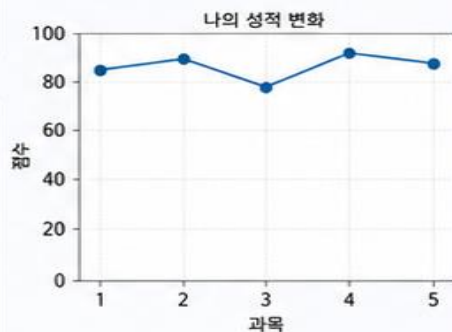
```
min_score = min(scores)
```

④ 결과 출력

```
print("평균:", avg)
print("최고점:", max_score)
print("최저점:", min_score)
```

⑤ 그래프 그리기

```
import matplotlib.pyplot as plt
plt.plot(scores, marker='o')
plt.title('나의 성적 변화')
plt.xlabel('과목')
plt.ylabel('점수')
plt.ylim(0, 100)
plt.grid(True)
plt.show()
```



실습 결과 작성

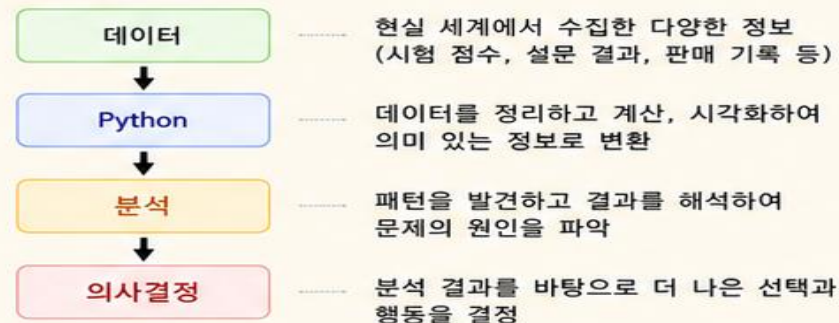
항목	결과
평균	
최고점	
최저점	



생각해 보기

- 데이터가 많아지면 사람이 직접 계산할 수 있을까?
- Python은 왜 필요할까?

★ TIP



참고

필요한 경우 `round()`, `sorted()`, `len()`, `min()`, `max()`, `sum()` 등을 활용해 보세요!